

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality.
(BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I Aryan.KS / 192325051 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in black ink, appearing to read 'Aryan', is written on a light gray rectangular background.

QUESTIONS:

Total Marks:25

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

- Identify the conceptual error in the likelihood formulation.
- Explain how this mistake affects parameter estimation.
- Suggest the correct mathematical formulation and reasoning.
- How would using log-likelihood fix computational issues?

ANSWER:

Error Analysis in Maximum Likelihood Estimation (MLE)

a) Conceptual Error in the Likelihood Formulation

The student incorrectly writes the likelihood as the **sum of probabilities** instead of the **product of probabilities**.

Incorrect Formula:

$$L(\theta) = \sum_{i=1}^n P(x_i | \theta)$$

Correct Formula:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

The likelihood represents the probability of observing the **entire dataset**, assuming all observations are independent. Therefore, probabilities must be multiplied, not added.

b) Effect on Parameter Estimation

Using the sum instead of the product does not correctly represent the joint probability of the data. As a result:

- The estimated parameters will not maximize the true likelihood.
- The optimization process converges to incorrect values.
- Even with large datasets, the model produces biased and inaccurate estimates.

Thus, the classifier performs poorly because the learned parameters do not represent the underlying data distribution.

c) Correct Mathematical Formulation and Reasoning

For independent observations,

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

The Maximum Likelihood Estimate is

$$\hat{\theta} = \arg \max_{\theta} L(\theta)$$

Since multiplying many probabilities gives extremely small values, we maximize the log-likelihood instead.

For a Gaussian distribution,

$$L(\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i-\mu)^2}{2\sigma^2}}$$

The MLE estimate of the mean is

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$

This estimator maximizes the probability of observing the given data.

d) How Log-Likelihood Fixes Computational Issues

Log-likelihood is

$$\log L(\theta) = \log \left(\prod_{i=1}^n P(x_i | \theta) \right)$$

Using logarithm,

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

Advantages:

- Prevents numerical underflow.
- Converts multiplication into addition.
- Makes differentiation simpler.
- Faster and more stable optimization.

Hence, almost all machine learning algorithms maximize log-likelihood instead of likelihood.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

- a) List possible implementation errors in the learning algorithm.
- b) Analyze how incorrect learning rate or weight updates affect convergence.
- c) Identify dataset-related issues that may cause this problem.
- d) Propose modifications to ensure convergence.

ANSWER:

Error Diagnosis in Perceptron Learning

a) Possible Implementation Errors

Common implementation mistakes include:

- **Incorrect weight update equation.**
- **Wrong learning rate.**
- **Missing bias update.**
- **Incorrect label encoding.**
- **Wrong stopping condition.**
- **Incorrect prediction function.**

b) Effect of Incorrect Learning Rate or Weight Updates

Perceptron update rule:

$$w_{new} = w_{old} + \eta(y - \hat{y})x$$

where

- w = weight vector
- η = learning rate
- y = actual output
- $\hat{y}$ = predicted output

If the learning rate is too high:

- Weights overshoot the optimal solution.
- Oscillations occur.

If it is too low:

- Convergence becomes extremely slow.

Incorrect update equations prevent the perceptron from finding the separating hyperplane.

c) Dataset-Related Issues

Possible reasons include:

- Labels are incorrect.
- Data is not truly linearly separable.
- Presence of noisy samples.
- Missing normalization.
- Duplicate conflicting samples.

These issues prevent successful convergence.

d) Modifications to Ensure Convergence

Use:

- Correct update rule.
- Appropriate learning rate.
- Normalize input features.
- Include bias term.
- Shuffle training samples.

Perceptron Convergence Theorem states that for linearly separable data, the perceptron always converges.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

- Identify possible mathematical or coding errors in gradient computation.
- Explain the role of activation functions in gradient flow.
- Analyze how vanishing or exploding gradients affect training.
- Suggest techniques to fix the issue (e.g., normalization, initialization).

ANSWER:

Backpropagation Gradient Error Analysis

a) Mathematical or Coding Errors

Common mistakes:

- Wrong derivative calculation.
- Wrong chain rule implementation.
- Incorrect learning rate.
- Updating weights before computing gradients.
- Incorrect sign during gradient descent.

Gradient descent update:

$$w = w - \eta \frac{\partial L}{\partial w}$$

Any mistake in the gradient causes increasing loss.

b) Role of Activation Functions

Sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Derivative:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Activation functions introduce non-linearity.

Poor activation functions can produce very small gradients, slowing learning.

ReLU:

$$f(x) = \max(0, x)$$

helps reduce vanishing gradients.

c) Vanishing and Exploding Gradients

Vanishing Gradient:

$$\frac{\partial L}{\partial w} \approx 0$$

Effects:

- Very slow learning.
- Earlier layers stop updating.

Exploding Gradient:

$$\frac{\partial L}{\partial w} \rightarrow \infty$$

Effects:

- Extremely large updates.
 - Training becomes unstable.
 - Loss increases rapidly.
-

d) Techniques to Fix the Problem

Use:

- Xavier Initialization
- He Initialization
- Batch Normalization
- Gradient Clipping
- ReLU activation
- Adam Optimizer

These methods stabilize gradients and improve convergence

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

- Explain why batch gradient descent is slow in large datasets.
- Analyze the cause of fluctuations in SGD.
- Compare the error behavior of Mini-Batch Gradient Descent.
- Recommend the best approach and justify with error trade-offs.

ANSWER:

Gradient Descent vs SGD Error Investigation

a) Why Batch Gradient Descent is Slow

Batch Gradient Descent computes gradients using the entire dataset.

Formula:

$$w = w - \eta \frac{1}{n} \sum_{i=1}^n \nabla L_i$$

For large datasets:

- Every iteration processes all samples.
- Computational cost becomes high.
- Training becomes slow.

b) Cause of Fluctuations in SGD

SGD update:

$$w = w - \eta \nabla L_i$$

where only one sample is used.

Advantages:

- Faster updates.
- Less computation.

Disadvantages:

- High variance.
- Noisy gradients.
- Loss fluctuates during training.

c) Mini-Batch Gradient Descent

Mini-batch update:

$$w = w - \eta \frac{1}{m} \sum_{i=1}^m \nabla L_i$$

where

m is the batch size.

Advantages:

- Faster than Batch GD.
- More stable than SGD.
- Better convergence.
- Efficient GPU utilization.

d) Best Approach

Mini-Batch Gradient Descent is generally preferred because:

- Faster convergence.
- Lower computational cost.
- Reduced gradient variance.
- Stable optimization.
- Better generalization.

It provides a balance between Batch Gradient Descent and SGD

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

- a) Explain how high dimensionality increases model error.
- b) Identify signs of overfitting in this scenario.
- c) Analyze why distance-based learning fails in high dimensions.
- d) Suggest dimensionality reduction or regularization techniques to improve performance.

ANSWERS:
Curse of Dimensionality Error Analysis

a) How High Dimensionality Increases Error

As dimensions increase:

- Data becomes sparse.
- More parameters must be learned.
- Risk of overfitting increases.

Distance formula:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

In high dimensions, distances between points become nearly identical, reducing the usefulness of distance metrics.

b) Signs of Overfitting

Typical signs include:

- Very high training accuracy.
 - Low test accuracy.
 - Large gap between training and validation error.
 - Model memorizes training data instead of learning patterns.
-

c) Why Distance-Based Learning Fails

Algorithms like KNN rely on Euclidean distance.

Formula:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

In high-dimensional space:

- Distances become less meaningful.
- Nearest neighbors are no longer truly similar.
- Classification accuracy decreases.

This phenomenon is known as **distance concentration**.

d) Techniques to Improve Performance

1. Principal Component Analysis (PCA)

Projects data into lower dimensions.

Covariance Matrix:

$$C = \frac{1}{n} X^T X$$

PCA retains components with the highest variance while removing redundant features.

2. L1 Regularization (Lasso)

$$Loss = MSE + \lambda \sum |w|$$

Removes unimportant features by driving weights to zero.

3. L2 Regularization (Ridge)

$$Loss = MSE + \lambda \sum w^2$$

Prevents excessively large weights and reduces overfitting.

4. Feature Selection

Methods such as mutual information, correlation analysis, and recursive feature elimination remove irrelevant features, reducing dimensionality and improving model generalization